EECS 4314

Project Assignment #1:
Conceptual Architecture Report

Group F
Sachin Kumar
Hardik Arora
Ahmed Elnahass
Minseo Kim
Mark Susanto

1. Overview

1.1 What OpenPilot is

OpenPilot is an open source driver assistance system made by comma.ai. It runs on small devices that sits on thes windshield of the car and connects to the car's own network. From there , it helps the driver by controlling the steering, the gas and the brakes. The GitHub page say it currently works with more than 300 supported cars. It also describes OpenPilot as "an operating system for robotics", which suggests the designers want the system to be useful beyond cars in the future.

OpenPilot does not make the car completely self driving. The driver stay responsible and has to be ready to take over at any time. This is one of the main ideas behind the whole design.

1.2 What it does

From the documentation, the main features are:

- Adaptive Cruise Control (ACC): keep set speed and slows down to stay behind the car in front.
- Automated Lane Centering (ALC): keeps the car in the middle of its lane.
- Lane change assist: helps car move into an another lane when the driver requests..
- Driver monitoring: uses a camera facing the driver to check that they're paying attention.

In this report we also look at Forward Collision Warning (FCW), which warns the driver when the car may hit the vehicle ahead. We chose ALC and FCW for our sequence diagrams because both involve many parts of the system working together.

1.3 Why it was built this way

The SAFETY documentation sets two rules that we think shape much of the architecture. The driver must always be able to take control right away, and the system must be incapable of changing the car's path faster than a person could react to. A lot of the design choices we describe later, such as the separate safety hardware (the panda) and the driver monitoring, come from these two rules.

1.4 The system at a high level

At a high level, OpenPilot is a chain of stages. Cameras and sensors collect information. A neural network (the driving model) uses it to understand the road. Planning decides where the car should go, control work out the steering and speed needed, and a vehicle interface sends the commands to the car. The car's state is then fed back to system so it is a complete closed loop. The panda sits between the device and the car and checks every command before it goes through.

The parts do not call each other directly. According to the Cereal documentation, they pass messages by using a publish and subscribe system, so each part only needs to know the message format and not who is on the other side. That is the main reason the system is easy to split into separate parts. We use this idea throughout the report.

1.5 How we studied it

We built our conceptual architecture mainly from the documentation: the OpenPilot website, the GitHub page, the Cereal documentation and the SAFETY documentation. Where the documentation does not give details, such as the exact lists of processes or how fast each one runs we made assumptions and we say so in the text. We also used GenAI to compare our diagrams with ones it produced. How we used it, and what we learned, is described in section 8.

1.6 How this report is organized

Section 2 presents the conceptual architecture and the alternative styles we considered. Section 3 describes each subsystem and what it does. Section 4 explains the ALC and FCW use cases with sequence diagrams. Section 5 covers control and data flow, and section 6 covers concurrency. Section 7 discusses how developer responsibilities could be divided. Section 8 explains how we used GenAI, and section 9 concludes.

2. Conceptual Architecture

2.1 Proposed architecture: five layers around a pipe-and-filter driving pipeline

Figure 1 illustrates how we model OpenPilot's conceptual architecture with five layers. Layer 3, which is the Driving Pipeline, is a pipe-and-filter system that changes sensor data into driving commands. Typed publish/subscribe message streams are the main method of communication between every layer, and the pipeline closes a feedback loop through the vehicle. Garlan and Shaw portray that real systems usually integrate styles. We utilize two of them since they answer two different questions. The layered figure depicts how the system is structured and what depends on what. The pipe-and-filter figure depicts how data progresses while the car is driving.

Figure 1 illustrates **five layers** from Vehicle and Hardware at the bottom to User and Services at the top through Platform, Driving Pipeline, and Supervision, and Section 3 will explain such layers. Every layer depends on the services of the layers under it. Supervision is positioned above Driving Pipeline since it reads the pipeline's outputs and transfers an engage or disengage decision down to Control.

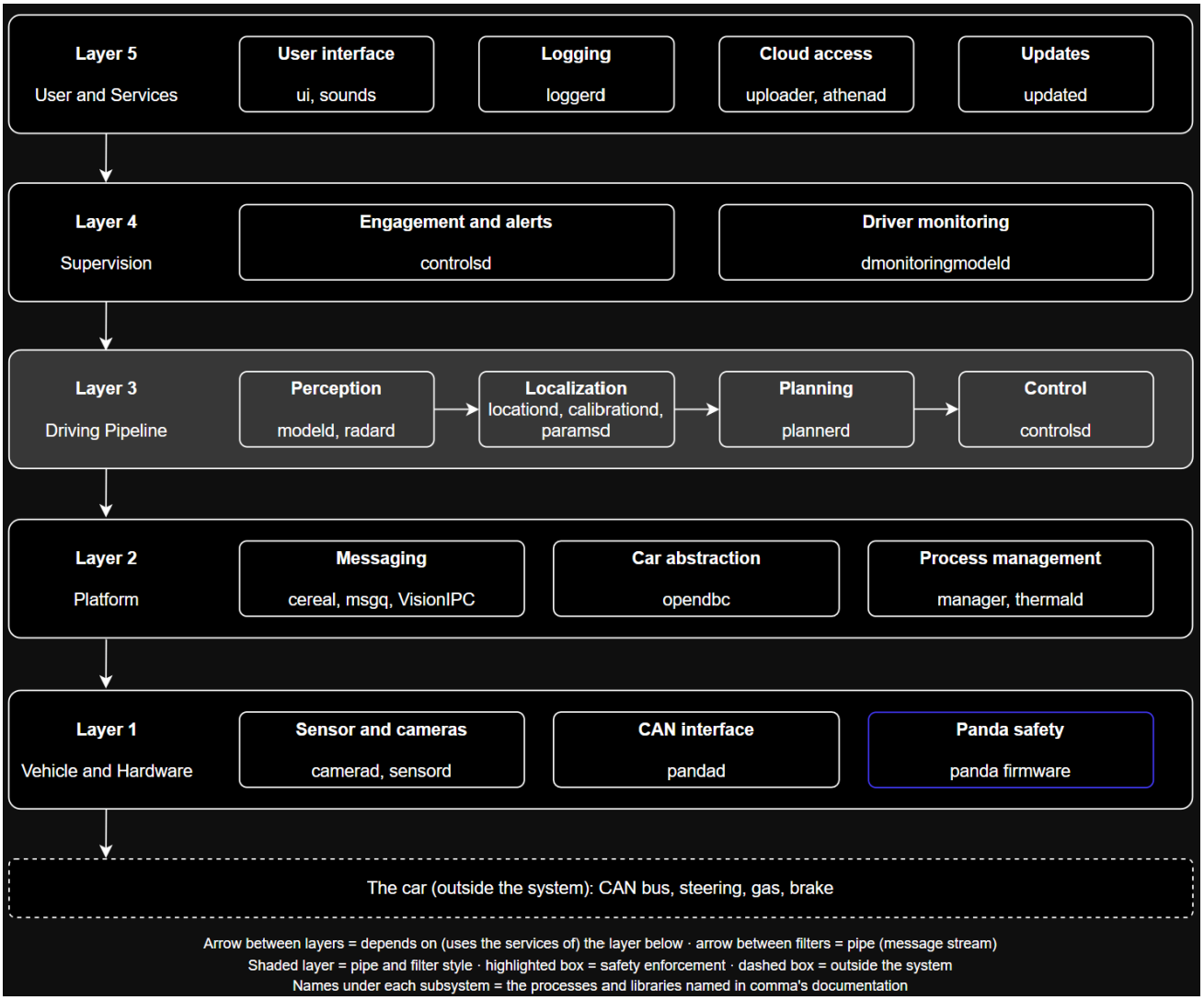


Figure 1: Conceptual architecture and dependencies, derived from the documents

Reading Figure 1 from the top to the bottom illustrates how each layer depends on the layers below it, and the car is positioned outside the system. The **pipe-and-filter** core is depicted by the shaded Driving Pipeline, and Figure 2 portrays how data flows through it.

Each element **inside the Driving Pipeline** is a filter that publishes a new stream through reading a continuous stream and then applying one local transformation. Perception utilizes camera frames and changes them to a driving plan and a prediction of the car's actions in front of the driver, which are then transformed into target speed and path by Planning. Control transforms the plan into commands that can be accepted by the car, and the invariants of the style persist. Filters do not change their states, and a publisher does not know who subscribes, which results in filters being unfamiliar with their neighbors (cereal README) (2019 tour). Therefore, all stages consecutively work on sequential moments of the drive since each filter is considered a separate process. The loop then closes through the car's response returning as CAN data and new sensor. Through this, such pipeline in general can also be considered a process-control system since the car is the process being controlled with the desired path and set speed being the set points.

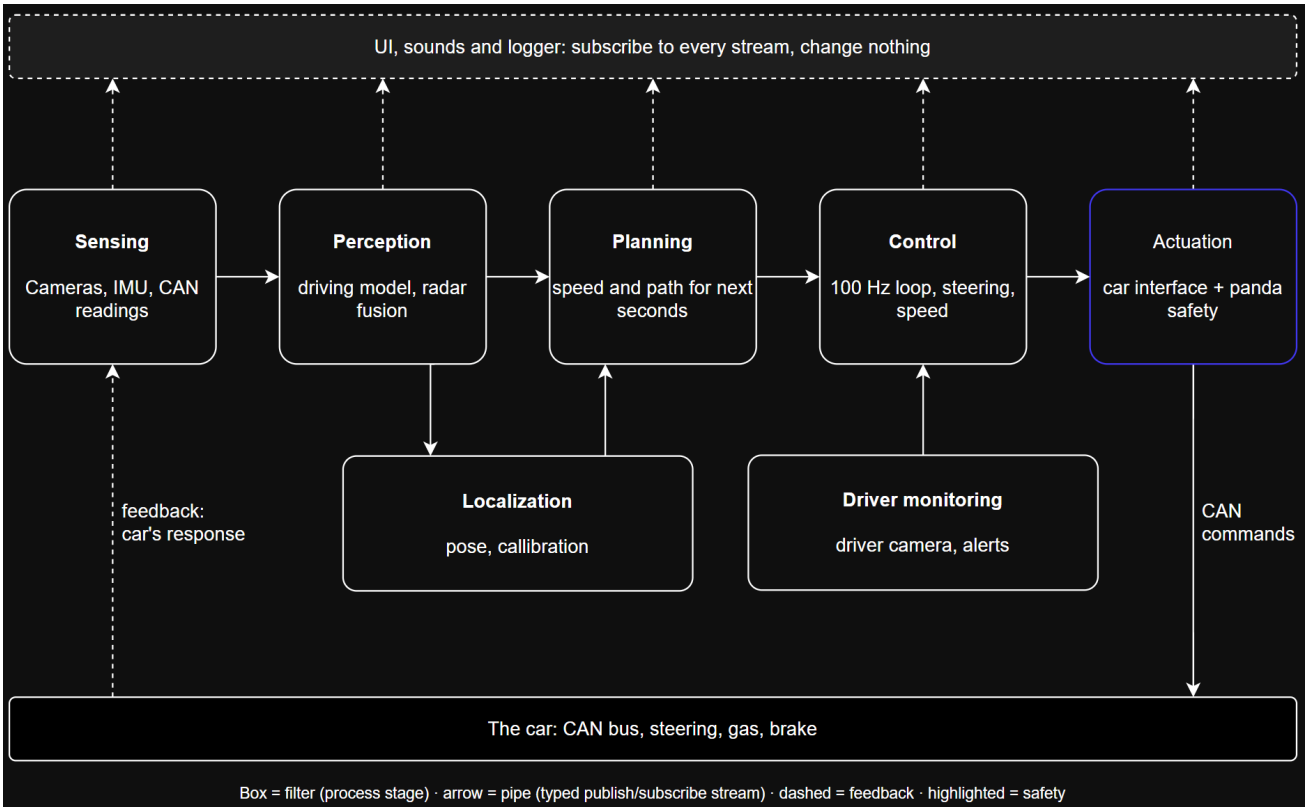


Figure 2: Data-flow perspective of the Driving Pipeline and its feedback loop

Figure 1 portrays how the system is organized, while Figure 2 portrays how data moves through such system while driving. The stream starts in Layer 1, passes through the Driving Pipeline, and ends at Actuation.

Each stream in the **publish/subscribe message streams** has one publisher and any number of subscribers. Messages use Cap'n Proto to be typed and utilize SI units. Backward compatibility is also in their format, hence old logs remaining readable (cereal README). Since camera frames are excessive in size, they move through VisionIPC, which is a shared-memory channel (2021 tour). The user interface and logger can listen to every stream without affecting driving since subscribing never changes a stream.

There are three interfaces that **further express the design**: the message specification is what connects all components; the car abstraction transforms any supported car into one CarState, which illustrates what the car is doing, and accepts one CarControl, which illustrates what the car should do (car port guide); and the panda safety model compares every exiting command against the limits of every car (opendbc safety model).

Our chosen conceptual architecture **satisfies OpenPilot's requirements** through achieving several factors: safety, performance, evolvability, and testability.

- **Safety**: First, our architecture enforces two requirements at the lowest layer, on a separate chip, set by SAFETY.md: the driver can immediately take back control at any time, and the car never changes its path faster than the driver can react. Through this, faults in the layers above won't be able to bypass the panda's safety checks.
- **Performance**: Moreover, performance is crucial for the path from camera to the CAN bus through perception, planning, and control. The control loop runs at 100 Hz (2021 tour), and the most intensive computation on the device is the neural network. This is clearly evident in

the statements “Reduced UI and modeld lags” and “Synchronize controlsd to pandad to reduce latency” (RELEASES.md). Instead of copying frames, pipe and filter moves data between filters through sharing such frames, hence portraying pipe and filter’s cost of moving data.

- **Evolvability**: Furthermore, a filter or a layer can be replaced as long as its interface doesn’t change, and the release notes illustrate both requirements. The entire messaging layer was changed without redesigning the components above it, where zmq was replaced by the custom messaging library msgq. End-to-end planning from the driving model also replaced the handwritten steering planner (RELEASES.md, version 0.10). A car interface would be added in the Platform layer without altering the pipeline if a new car is supported (car port guide).
- **Testability**: Finally, the logs contain recorded information regarding all messages passed between parts (logs). Filters can be utilized through replaying recorded drives into them, hence being “a critical tool for openpilot development and debugging” (replay guide). A dedicated safety test suite is kept by comma through performing “software-in-the-loop, hardware-in-the-loop, and in-vehicle tests before each software release” (SAFETY.md).

2.2 Alternatives considered

We had several candidates for our main conceptual architecture, but only the combination of the two architectures in Section 2.1 achieved our requirements and was able to answer these two questions: does it describe how the whole system is organized, and does it describe how data moves while driving? Our initial model was microservices, but our presentation only portrayed the pipeline, which can be seen in Figure 2, and reasoned for pure pipe and filter. The **feedback** we received illustrated how our overall structure was hard to follow, hence adding a layered architecture, which can be seen in Figure 1, around it without altering the pipeline itself.

Alternative	**Why we considered it**	**Why we rejected it**	**Where it still appears**
Microservices	The documents call the parts “services”. Each part is small, has one purpose, and started and stopped on its own (2021 tour).	Microservices are individually deployed, versioned, and scaled, and communicate via a network. OpenPilot is considered a monolithic release that is deployed onto one device, and its parts exchange local messages. If there exists a latency the 100 Hz control loop can’t afford, network calls would add such latency.	It appears nowhere since “service” in the documents means a process, not a microservice.
Pure pipe and filter	The Driving Pipeline is a constant stream transformed phase by phase.	It only encompasses the driving chain but not the parts outside it, such as the user interface, logging, cloud upload, and updates. It also doesn’t have any place for supervision, which watches the pipeline instead of being positioned in it.	The style of the Driving Pipeline layer.

Pure layered	Components evidently rely on lower-level services, from the hardware up.	Layers depict what relies on what. However, the feedback loop and the stream of data are hidden through the car, which are crucial to how OpenPilot operates.	The overall architecture (Figure 1).
Implicit invocation (publish/subscribe) as the main style	Every part communicates by publishing and subscribing to messages.	It expresses how components connect and not what they do. How the driving chain is ordered is unchangeable, portraying how there are no loose set of parts to react to arbitrary events.	The connector utilized by every layer.

Three more styles occur only in part. Process control depicts the driving loop together with the car, but it doesn't express supervision, the user interface, or logging. The logs are utilized like a repository for replay and analysis. However, parts don't move through them while driving, and only the link between the device and the cloud is client-server (2019 tour).

3. Subsystems and Functionalities

This section describes each subsystem in Figure 1, layer by layer from the bottom to the top. Names in parentheses are the processes identified by comma's documentation, and each runs as its own process.

3.1 Vehicle and Hardware layer

Sensors and camera drivers (camerad, sensord): Capture the cameras that are wide-angled, facing the driver, and facing the driver. They also capture the motion sensors and GPS and then publish everything for the layers above (2019 tour).

CAN interface (pandad, formerly boardd): Transfers CAN messages between the device and the car (RELEASES.md). The car's speed, steering angle, pedals, and buttons are given by the reading to OpenPilot. Steering, gas, and brake commands are delivered by the writing through "the car's existing APIs for the built-in ADAS system" (docs.comma.ai).

Panda safety firmware: A specific chip between the device and the car that compares every exiting command against the limits of every car, blocks commands while OpenPilot is disengaged, and returns control when the driver brakes or cancels (opendbc safety model) (SAFETY.md). It can block commands even if the rest of the system fails since it is the final check before anything reaches the car.

3.2 Platform layer

Messaging (cereal, msgq, VisionIPC): Identifies every message type and transfers messages between processes, from one publisher to several subscribers, and camera frames go through shared memory (cereal README) (2021 tour).

Car and radar abstraction (opendbc car interfaces): Transforms every car's own radar and CAN messages into one comprehensive CarState and transforms CarControl back into that car's messages, hence portraying how one pipeline can drive more than 300 car models (car port guide) (supported cars).

Process management (manager, thermald): The manager starts and stops processes based on whether the car is on or off the road. Device health and temperature are monitored by thermald, and it determines when it is safe to go back on the road (2021 tour).

3.3 Driving Pipeline layer

Perception (modeld, radard): The driving model receives camera frames and a "desire" input, such as a requested lane change. It calculates where the lead cars are and where the car should drive (2021 tour). Such operation is shared by a driving policy and a vision model that are trained in a simulator, and lane lines are used only for display(OpenPilot 0.11). Radar tracks are combined with the model's prediction of the car's actions in front of the driver by radard.

Localization (locationd, calibrationd, paramsd): locationd estimates orientation, velocity, and position through integrating camera motion with inertial sensors in a Kalman filter. calibrationd figures out how the device is mounted. paramsd learns values that are specific to each car, such as tire stiffness and steering ratio (2021 tour).

Planning (plannerd and the driving model): Determines the next few seconds of action or motion. The steering path is planned by the driving model through direct images from the camera (RELEASES.md, 0.9.5). Speed follows the set speed and the lead car or follows the model's own plan when in Experimental mode (OpenPilot 0.11).

Control (controlsd): A 100 Hz closed loop that sends the acceleration and steering commands to the car abstraction as CarControl through utilizing the target speed and path.

3.4 Supervision layer

Engagement and alerts (part of controlsd in the documents): Determines if OpenPilot is able to engage, remain engaged, or should disengage. It reacts to the driver's commands, such as the buttons being pressed, brake, and gas, and it reacts to errors such as a failed sensor. The driver's alerts are also raised by it.

Driver monitoring (dmonitoringmodeld and its policy): A separate model observes the driver camera for eye state and head pose. When the driver is sleepy or distracted, alerts get escalated by the policy. It can also disengage OpenPilot (docs.comma.ai) (2021 tour).

3.5 User and Services layer

User interface (ui, sounds): Portrays the driving view, the planned path, set speed and alerts, and plays alert sounds.

Logging (loggerd): Records every camera stream and message in segments that are one minute long, and these include full logs (rlog), reduced logs (qlog) and video (logs).

Cloud access (uploader, athenad): Uploads logs, deletes old data, and allows the owner to get access to the parked car through an API (2019 tour).

Updates (updated): Installs new OpenPilot versions over the air (2019 tour).

4. Use Cases and Sequence Diagrams

4.1 ALC (Automated Lane Centering)

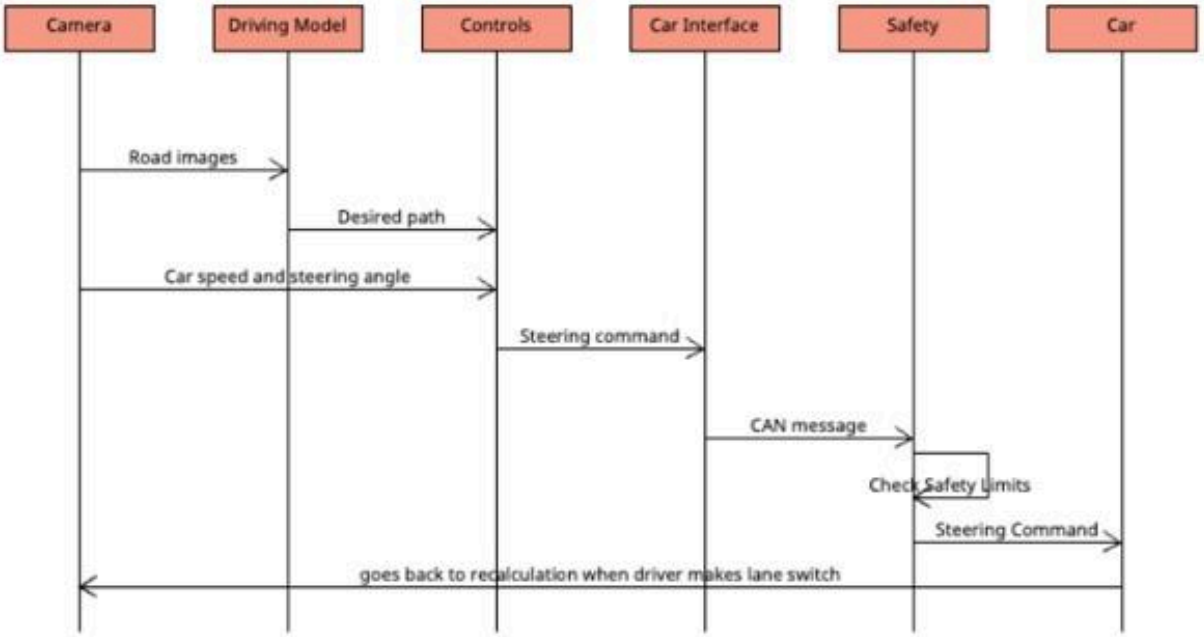


Figure 3:Sequence Diagram for ALC (Drawn with UMLetino)

One of the key functionalities is ALC (Automated Lane Centering), which is basically the system keeping the car centered in lane. The reason we chose to draw sequence diagrams for ALC and FCW is because they both have lots of microservices working together and communicating with each other and although they're just small components it represents how the overall openpilot works as a whole. As you can see in the diagram, everything start with the camera data. The camera captures road and sends the input data to the driving model. We found this from modeld.py where the run function in ModelState takes in the camera frames. Since we looked at this at a high level, we didn't go into every detail of how the frames are processed. So now that the data is in driving model, we now have to know what the driving model does. On the website, we noticed that openpilot uses a neural network that learned to drive from millions of driving data. When we looked at the code, we noticed that modeld.py uses tinygrad, a deep learning framework.Inside that same run function, the road images are fed into the neural network, and the neural network predicts the path the car should follow.Since we can't really handle raw output directly get_action_from_model turns the prediction into a desired path and send it to the controls. Once it reaches the controls section, they compare the desired path output from model section and the current speed and angle, then it sends out a command to car interface. Because every car brand understands the command differently, card.py is where it calls the specific car's information and function and translates the command into the form that car can understand. This whole loop keeps running over and over while the openpilot is engaged so when the driver tries to change lane, it goes back to the start and recalculates a new path for new lane.

4.2 FCW (Forward Collision Warning)

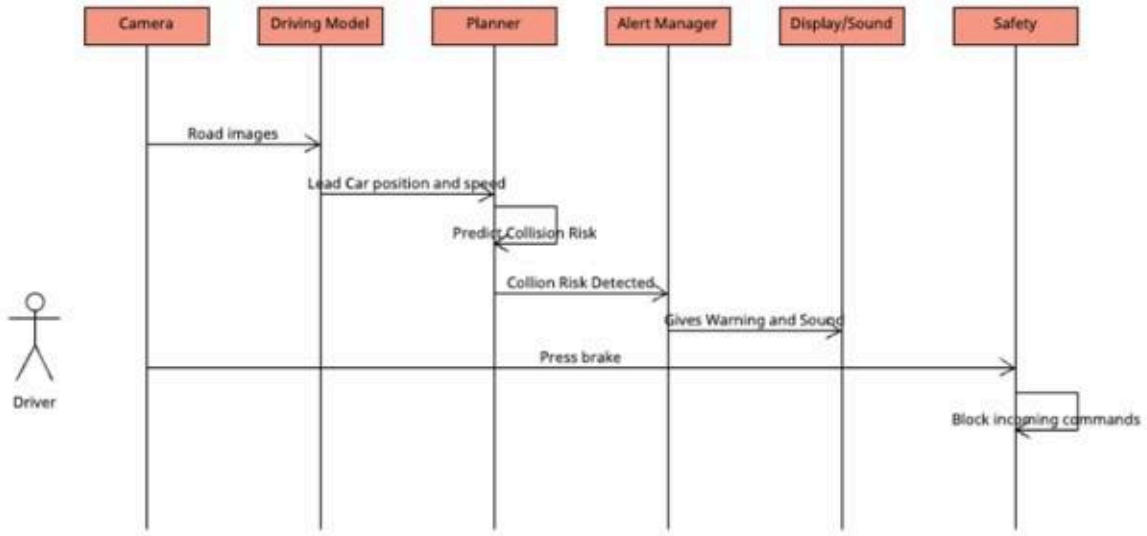


Figure 4:Sequence Diagram for FCW (Drawn with UMLetino)

Forward Collision Warning is another important feature that warns the driver when the car might crash into the car in front of you. Just like ALC, it starts with the input data from camera sending road images to the driving model through the same run function. The difference is what the model looks for. Instead of only predicting the path, it also detects the car ahead and predicts whether braking will be needed. In fill_model_msh.py, the model's braking predictions are checked over the last few frames, and if they stay high, it sets a flag called hardBrakePredicted. For this one, not only the road images, the radar data also comes in through the car interface. That lead car information then goes to the Planner. The planner's job is to plan the car's speed and as part of that, its update function predicts the distance to the lead car over the next few seconds. The obvious reason why this was not needed in ALC is because it has something to do with speed. When the distance gets too small not only it sets its own FCW flag. One thing we noticed about this system is that every parts of the system is very distinct and distributed in terms of their roles. So just because model and controls flag their own FCW, it doesn't directly got ot the car but it goes through selfdrived.py and events.py which also acts as an alert manager. If either the model or the planner flagged a risk, it gives an alert with a warning sound and it gets shown on the display.

4.3 How this connects to the architecture

Looking at both diagrams together, we can see the pipe and filter architecture clearly. Each component acts as a filter where they take data in, does one specific job and passes the result o the next component. The messages and connections between them are the pipes. Although ALC uses controls and FCW uses the planner, they bother start with the same pipeline where they receive the data and also pass though the panda at the end which is a final safety filter.

5. Control and Data Flow

At a conceptual level, the system essentially works as a chain of stages. Each stage reads the output of the previous one, completes its task, and publishes its result for the next stage.

End-to-End Flow

1. Data Collection. The camera and sensor processes collect information from the vehicle and its surroundings. They do not do any interpretation.

2. Perception. The Perception component processes this information to understand the road, such as lane lines, the path ahead, and lead vehicles.

3. Planning. Planning uses that understanding to determine the vehicle's intended movement path.

4. Control. The Control component calculates the steering and acceleration or braking commands needed to carry out the plan.

5. Vehicle Interface. The Vehicle Interface sends these commands back to the car.

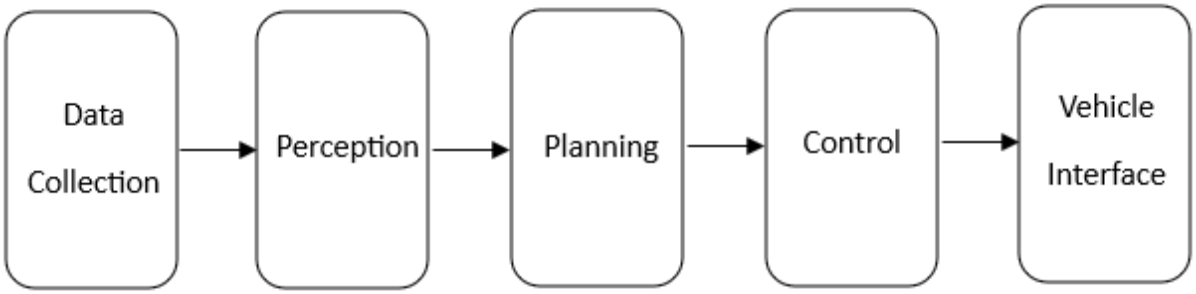


Figure 5: Conceptual control and data flow in OpenPilot

Communication Mechanism

The components exchange messages through OpenPilot's pub-sub system, Cereal. A component publishes messages on named topics, and any component that needs the data subscribes to that topic. Each component has its own responsibility and depends only on the message format.

Architectural Style

This flow resembles pipe-and-filter architecture, because information passes through different stages and each stage processes it for the next task.

The components can also be seen as forming a layered architecture. Data collection sits closest to the vehicle and its surroundings, perception and planning build on top of it to understand and decide, and the control and vehicle interface sit at the level of acting on the car. Each layer builds on the output of the one before it and has clearly separated responsibilities.

However, the flow is not strictly linear. Vehicle state information also feeds back into the system to update control decisions as the driving situation changes. This feedback forms a closed loop on top of the existing pipeline.

Overall, the components work together to process information and control the vehicle. Data moves from collection through perception, planning, and control to the vehicle interface, and vehicle-state feedback keeps control decisions current.

6. Concurrency in OpenPilot

6.1 What runs at the same time

Driving is a continuous job. The car do not stop and wait while OpenPilot thinks. At any moment the car is acting on a command from a moment ago while the system is already looking at the newest camera picture to work out the next command. To do this, many parts of the system have to work at the same time.

The documentation does not list the processes so this part is our own assumption. We think each major component run as its own separate process on the comma device. These are camera and sensor capture, the driving model, planning, controls, the car interface and driver monitoring. We assume that there is something that starts these processes when the car turns on and keeps them running.

6.2 How control moves from one component to another component?

The components do not call each other like functions in a simple program. The documentation says they pass messages through Cereal, the publish and subscribe system. One component publishes a message, and any other component who needs it receives it. So control moves when a message comes, but not when one component hands it over to another.

We also assume that each component works in one of two ways. Some wait for the input they need before doing anything. The driving model is a good example, since it has nothing to do until a new camera image arrives. Others repeat their work on a regular schedule and use the latest message they have. Controls is a good example, because steering needs frequent updates.

6.3 Different Components run at different speeds

Since every component has its own loop, they do not have to run at the same speed. We expect the camera and the driving model to be slower, because they depend on how quickly images come in and how long the neural network takes. We expect controls to be faster, since steering and braking need to feel smooth.

This works because a fast component can keep using the newest message from a slow one until a fresh one arrives. The downside is that a command may be based on information that is a little old. Every stage adds some delay, so the total delay from camera to steering matters a lot.

6.4 Concurrency in our use cases

Our ALC diagram shows the chain: camera, driving model, controls, car interface. At any moment each of these is busy with a different piece of data. The camera is already capturing a new image while the model works on the last one and controls act on the model's last result. This overlap is why the loop never stops at all.

The FCW diagram shows a different kind of concurrency. The driving model and the planner each checks for a possible crash on their own, in the separate parts of the system, and they don't need to talk to each other. They only meet at the alert part, which combines their warnings and tells the driver. Because they work independently, the warning doesn't depend on just one of them.

Driver monitoring is also independent. It keeps watching the driver whether or not the driving part is busy, and it affects the car only through the engagement logic.

6.5 Safety and failures

The panda is a separate piece of hardware between the device and the car. The SAFETY documentation says it enforces its own limits on what the car can be told to do. This means it runs in parallel with everything else and does not rely on the device's software behaving properly. If something on the device goes wrong, the panda is still there as a last check.

We also assume the rest of the system needs a way to notice when a component has stopped sending messages, because otherwise the others would keep using old data without knowing. This matches the "faulty system" arrow to Soft disabled in the self-drive diagram.

6.6 Problems that concurrency causes

- There is no single order in which everything happens. Parts run at the same time, so correct behaviour depends on each part doing its job on time.
- Delay adds up. If one stage is slow, everything after it works with older data.
- Heavy parts compete for the same device. The driving model is probably the heaviest, and it shares the device with logging and driver monitoring.
- Testing is harder, because a bug may only appear with a certain timing.

6.7 What this means for the architecture

Splitting the system into separate concurrent parts has real benefits. One part failing doesn't stop the whole system. Each part can be developed and tested on its own, as long as it keep the same message format. A part can also be replaced (for example a new driving model) without changing the others. The cost is the timing problems in section 6.6 which is why the most safety-critical limits sit in the panda and not in the software on the device. This also connects to section 7 of the report because each team can own a component and only has to agree on the messages.

7. Division of Developer Responsibilities

7.1 What aspects of this architecture impact development?

Like previously mentioned in our proposed style analysis, there is a clear separation between the many subsystems of the project. Although they each serve drastically different purposes and functions spanning a wide domain of specialities, they all have dependencies on one another. This was evident in the clear flow of data being transformed at the consequent layer until its effects on the physical components. This is unlike architectures such as repositories whose components don't influence one another outside the main central data structure. The following sections discussed how this implicates the creation and further maintenance of the system.

7.2 Developers are able to specialise in respective area

By properly placing boundaries for each component and creating these distinct and clear points of interaction between them, the developers are able to keep their work very contained. Since the system requires such widely differing fields of expertise such as hardware experts for the interface of

the vehicle or machine learning experts for the perception of the environment, it helps in proper scope control. After agreeing upon a proper software contract in the expectations of input and outputs, there is less consideration for the work of others outside of your own. Thus responsibilities can be distributed as long as extensive planning is performed.

7.3 Developers are able to work in parallel

The timeline is another aspect this style can influence. Like previously mentioned above, the clear cut boundaries enable less dependencies amongst developers. This enables multiple developments to be performed simultaneously. Although there is a caveat and that is the software contract. There is a lot of consideration in the planning phase to ensure that there is a consistent transformation flow of the data and that each step is achievable and reasonable for the subsystem. Like a black box the contents and how it is manipulated is not to be worried about as long as requirements are fulfilled as expected.

7.4 Deeper emphasis on testing and quality assurance

Due to the linear nature of the architecture, although mistakes and issues that arise in a component might not affect its direct neighbor in its output, it has the very real possibility of propagating down the chain of control and impact downstream systems. This implies that there must be more work done in ensuring that however the subsystem produces its result, it does it correctly. Thus extensive component testing must be done. But even when this passes, minor mistakes might only be evident far from the source so proper integration testing between many connectors and overall system wide simulations are needed. Especially with the overall impact if something like a vehicle control system fails. Thus other than the dedicated testing responsibility of the regular professionals there are needed specialists like a QA team dedicated to rigorously checking all points of failure.

8. Conclusion

8.1 What we learned

Analyzing the architecture for a big platform such as OpenPilot was harder than we expected. We initially hypothesized as a microservices but we realized the parts all run on one device and ship together. As a result, we ended up with layers around a pipe-and-filter driving pipeline. Initially, we thought that the architecture that would satisfy all requirements is a pure pipe and filter. However, feedback portrayed how such architecture was hard to follow, hence learning that there can’t be only one architecture that illustrates how the system is organized and how data moves through it. Therefore, we resulted to creating a layered view with a pipe-and-filter core to answer both questions and satisfy all requirements.

The most important thing we found is that safety rules have the greatest influence on design. Because the driver must always be able to take over, there is a panda, driver monitoring, and engagement logic. The format of the message is also very crucial. The components can operate simultaneously, be tested by replaying logs, and be modified without coming into contact with one another because they only agree on these messages. This also makes it easier to observe how a team divides their work.

Furthermore, one of the big limitations we faced was the lack of low level details. Since we can’t read and analyze every single codebase, it was quite difficult to understand how everything was

interconnected. Because of this reason, our diagrams are shown to be very high level and doesn't contain detailed flow. To experiment, we have tested with genAI to see how well it performs and how it could benefit our research on openpilot. Below is our prompt on claude and diagrams we received from it.

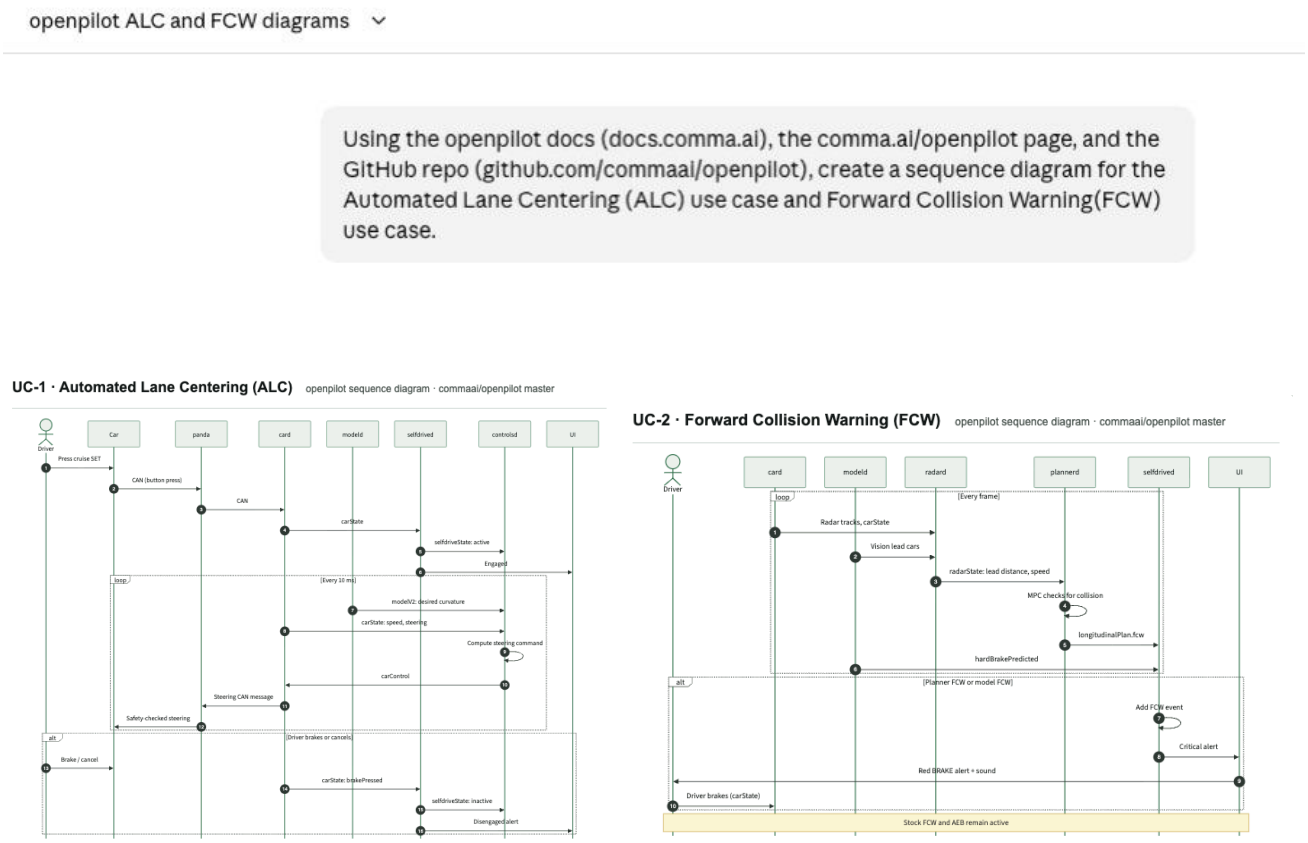


Figure 6: Prompt and diagrams generated with Claude

As you can see, the AI's diagrams were much closer to the actual code. They used real process names and included steps that we left out including driver engaging openpilot, radar data being combined with the camera data. However, both diagrams also had gaps. Neither diagram included the camera so the driving model receives no input. Overall, AI was useful for quickly mapping our high level components to the actual code and for spotting our own mistakes but its output still had to be checked as stated above. Al may be more detailed in terms of code similarity but it can never catch the nuance unless we give very detailed prompts or repetitive prompt engineering. Our simpler diagrams were better for explaining the architecture.

8.3 Source of information

- **OpenPilot Github Page: https://github.com/commaai/openpilot**
- **OpenPilot Website: https://www.comma.ai/openpilot**
- **OpenPilot's Blog Website: https://blog.comma.ai/a-tour-through-openpilot/**
- **OpenPilot's Docs Website: https://docs.comma.ai/**